# Project Report

**Title: <u>DNA Sequence Analyzer</u>**

**Submitted by: Siddiqa Rani**

**Submitted to: Ma'am Momina Jabeen**

**Introduction**

DNA Sequence analysis is a fundamental operation in bioinformatics. This project develops a Python-based tool to perform multiple DNA sequence operations such as GC content calculation, reverse complement, transcription to RNA, translation to protein, base compositio n plotting, and pairwise sequence alignment (global or local).

The program is fully interactive, offering both manual input and automated processing. It uses BioPython and Matplotlib libraries for sequence manipulation and visualization.

**Objectives:**

The objective of this project is:

- To analyze DNA sequences for basic biological features.
- To visualize DNA base composition.
- To align two sequences and understand their similarities/differences.
- To design a modular and expandable bioinformatics tool using Python.

**Tools & Libraries Used:**

| | Library | Purpose |
|---|---|---|
| 1. | **matplotlib** | To plot DNA base composition and sequence align ment |
| 2. | **Bio.Seq** | To work with DNA sequences |
| 3. | **Bio.pairwise2** | To perform pairwise sequence alignment |
| 4. | **Bio.pairwise2.format_alignment** | To display aligned sequences neatly |

**Methodology:**

**i. Sequence Input:**

User is asked to enter two DNA sequences manually. Sequences are cleaned by removing spaces and converting to uppercase. Then the user is asked to combine both sequences or not, if yes both the sequences are combined, and user can analyse it further. This choice d efines com_seq, which is used for most operations.

```python
print("\n--- Welcome to DNA Sequence Analyzer ---\n")

print("---Welcome to dna sequence analyzer---")
seq_1=input("enter first sequence: ").upper().replace(" ","").replace("\n","")
seq_2= input("enter ssecond sequence: ").upper().replace(" ","").replace("\n","")
print("Your sequences are: \n",seq_1,"\n",seq_2,"\n")
combine_choice = input("Do you want to combine sequences for analysis? (yes/no): ").lower()
```

```python
if combine_choice == "yes":
    com_seq = seq_1 + seq_2
else:
    selected = input("Which sequence do you want to analyze? (1 or 2): ")
    if selected == "1":
        com_seq = seq_1
    else:
        com_seq = seq_2
```

```
---Welcome to dna sequence analyzer---
enter first sequence: atacgcgcgtatat
enter ssecond sequence: atgcatgcatgc
Your sequences are:
 ATACGCGCGTATAT
 ATGCATGCATGC
```

**II. Sequence Validation:**

Function validate DNA (seq) ensures sequences only contain valid bases: 'A', 'T', 'C', 'G'. If invali d bases are detected, the program prompts the user to re-enter.

```python
def validate_dna(seq):
    for base in seq:
      if base not in ['A','T','C','G']:
         return False
    return True
```

**III. Menu-driven Analysis:**

The user is shown a menu to select the operation:

```python
    while True:
        print("\nChoose an option:")
        print("1. Calculate GC Content")
        print("2. Find Reverse Complement")
        print("3. Transcribe DNA to RNA")
        print("4. Translate DNA to Protein")
        print("5. Plot Base Composition")
        print("6. Align Sequences (Global or Local)")
        print("7. Run Full DNA Analyzer (All Steps)")
        print("0. Exit")
```

```python
while True:
    print("Menu...")
    choice = input("""Please choose an option: \n1.Calculate GC Content\n
    2.Find Reverse Complement\n3.Transcribe DNA to RNA\n4.Translate DNA to Protein\n
    5.Plot Base Composition\n6.Align Sequences (global/local)\n
    7.Run Full DNA Analyzer (All Steps)\n0.Exit:""")

    if choice == "1":
        print("\nGC Content: ",calculate_gc_content(com_seq),"\n")

    elif choice == "2":
        print("\nReverse Complement: ",reverse_complement(com_seq),"\n")

    elif choice == "3":
        print("\nTranscribe DNA: ",transcribe_dna(com_seq),"\n")

    elif choice == "4":
        print("\nTranslate DNA",translate_dna(com_seq),"\n")

    elif choice == "5":
        plot_base_composition(com_seq)

    elif choice == "6":
        alignment_type = input("Choose alignment type (global/local): ").lower()
        align_sequences(seq_1, seq_2, alignment_type)
```

```python
elif choice == "6":
    alignment_type = input("Choose alignment type (global/local): ").lower()
    align_sequences(seq_1, seq_2, alignment_type)

elif choice == '7':
     print(f"\nGC Content: {calculate_gc_content(com_seq):.2f}%")
     print(f"\nReverse Complement:\n{reverse_complement(com_seq)}")
     print(f"\nRNA Transcription:\n{transcribe_dna(com_seq)}")
     print(f"\nProtein Translation:\n{translate_dna(com_seq)}")
     plot_base_composition(com_seq)
     alignment_choice = input("\nChoose alignment type (global/local): ").lower()
     align_sequences(seq_1,seq_2, alignment_type=alignment_choice)

elif choice == '0':
        print("\nExiting program. Goodbye!")
        break

else:
        print("\nInvalid choice. Please enter a number between 0 and 7.")
```

```
Menu...
Please choose an option:
1.Calculate GC Content
2.Find Reverse Complement
3.Transcribe DNA to RNA
4.Translate DNA to Protein
5.Plot Base Composition
6.Align Sequences (global/local)
7.Run Full DNA Analyzer (All Steps)
0.Exit:7
```

## Functional Details:

### 1. Calculate GC Content

The function calculate_gc_content(seq) counts G and C bases. Then Calculates the percentage of GC content relative to total length.

```python
def calculate_gc_content(com_seq):
  gc=com_seq.count('G')+com_seq.count('C')
  return gc/len(com_seq)*100
```

```
GC Content: 46.15%
```

**2. Find Reverse Complement:**

The function **reverse_complement(seq)** map the bases **(**A ↔ T**,** G ↔ C) and then reverses the sequence order.

```python
def reverse_complement(com_seq):
  complement={'A':'T','T':'A','G':'C','C':'G'}
  rev_comp=''.join(complement[base] for base in reversed(com_seq))
  return rev_comp
```

```
Reverse Complement:
GCATGCATGCATATATACGCGCGTAT
```

**3. Transcribe DNA to RNA**

Function used is **transcribe_dna(seq),** it replaces all 'T' bases with 'U' to create an RNA sequence.

```python
def transcribe_dna(com_seq):
  rna_seq=com_seq.replace('T','U')
  return rna_seq
```

```
RNA Transcription:
AUACGCGCGUAUAUAUGCAUGCAUGC
```

**4. Translate DNA to Protein**

**translate_dna(seq)** reads DNA sequence codon-by-codon (triplets). It maps codons to amino acids based on the standard genetic code. It also handles unknown codons gracefully with a '?'.

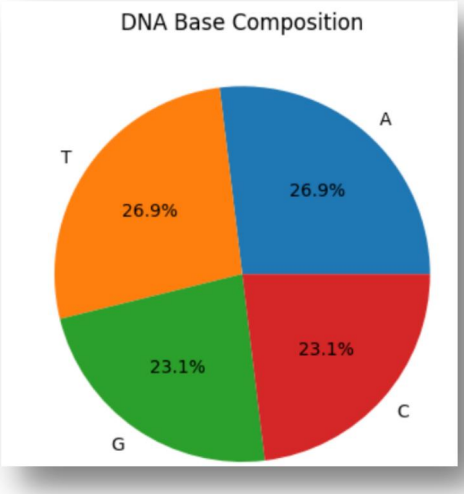
```python
def translate_dna(com_seq):
    codon_table = {
        'ATA':'I', 'ATC':'I', 'ATT':'I', 'ATG':'M',
        'ACA':'T', 'ACC':'T', 'ACG':'T', 'ACT':'T',
        'AAC':'N', 'AAT':'N', 'AAA':'K', 'AAG':'K',
        'AGC':'S', 'AGT':'S', 'AGA':'R', 'AGG':'R',
        'CTA':'L', 'CTC':'L', 'CTG':'L', 'CTT':'L',
        'CCA':'P', 'CCC':'P', 'CCG':'P', 'CCT':'P',
        'CAC':'H', 'CAT':'H', 'CAA':'Q', 'CAG':'Q',
        'CGA':'R', 'CGC':'R', 'CGG':'R', 'CGT':'R',
        'GTA':'V', 'GTC':'V', 'GTG':'V', 'GTT':'V',
        'GCA':'A', 'GCC':'A', 'GCG':'A', 'GCT':'A',
        'GAC':'D', 'GAT':'D', 'GAA':'E', 'GAG':'E',
        'GGA':'G', 'GGC':'G', 'GGG':'G', 'GGT':'G',
        'TCA':'S', 'TCC':'S', 'TCG':'S', 'TCT':'S',
        'TTC':'F', 'TTT':'F', 'TTA':'L', 'TTG':'L',
        'TAC':'Y', 'TAT':'Y', 'TAA':'_', 'TAG':'_',
        'TGC':'C', 'TGT':'C', 'TGA':'_', 'TGG':'W',
    }
    protein = ""
    for i in range(0, len(com_seq) - 2, 3):
        codon = com_seq[i:i+3]
        amino_acid = codon_table.get(codon, '?')
        protein += amino_acid
    return protein
    print(translate_dna(com_seq))
```

```
Protein Translation:
IRAYICMH
```

**5. Plot Base Composition**

**plot_base_composition(seq)** calculates counts of 'A', 'T', 'G', and 'C'. Plots these in a pie char t using matplotlib and labels each base with its proportion.

```python
def plot_base_composition(com_seq):
    seq = com_seq.upper()
    a_count = seq.count('A')
    t_count = seq.count('T')
    g_count = seq.count('G')
    c_count = seq.count('C')
    labels = ['A', 'T', 'G', 'C']
    sizes = [a_count, t_count, g_count, c_count]
    plt.pie(sizes, labels=labels, autopct='%1.1f%%')
    plt.title("DNA Base Composition")
    plt.show()
```




**6. Align Sequences (Global or Local)**

Function **align_sequences(seq1, seq2, alignment_type)** align the sequence and allows user t o select:

- **Global alignment** (Needleman-Wunsch algorithm) — matches whole sequences.
- **Local alignment** (Smith-Waterman algorithm) — matches sub-sequences.

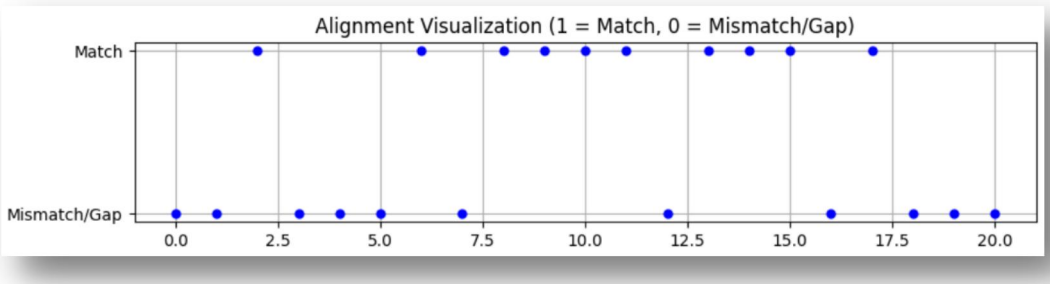
```
Choose alignment type (global/local): local
3 ATTTTCGCGCGGTGAC
  |   | |||| ||| |
1 A---T-GCGC-GTG-C
  Score=10
```

- Visualization:
  - Matches are plotted as blue dots at y=1.
  - Mismatches/gaps are plotted at y=0.

- A legend is also provided for clarity.

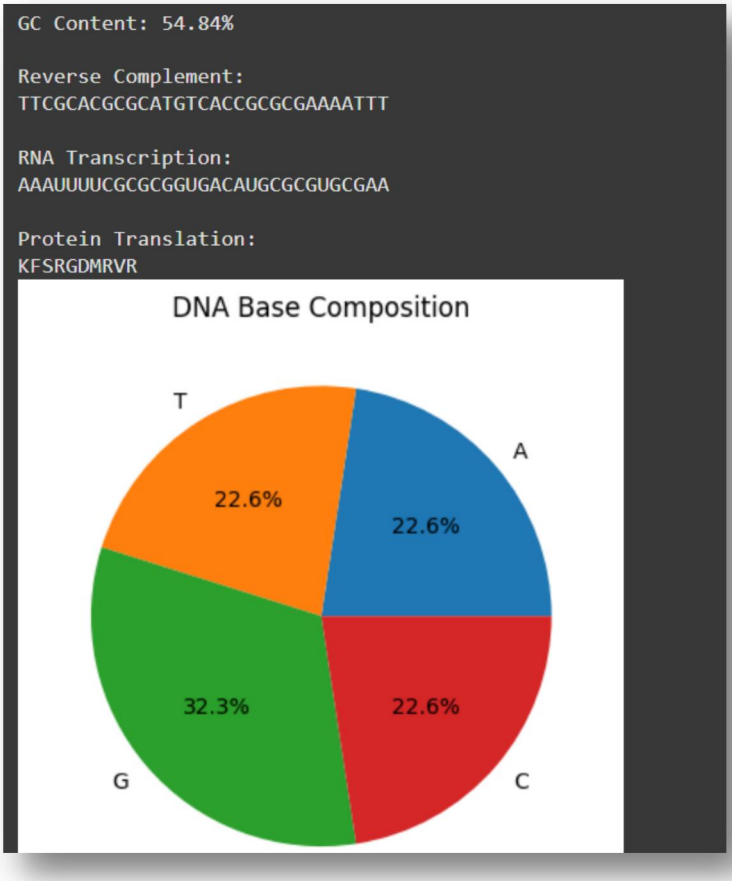
```python
def align_sequences(seq1, seq2, alignment_type):
    seq1 = seq1.upper()
    seq2 = seq2.upper()

    if alignment_type == "global":
        alignments = pairwise2.align.globalxx(seq1, seq2)
    else:
        alignments = pairwise2.align.localxx(seq1, seq2)

    best_alignment = alignments[0]
    aligned_seq1, aligned_seq2, score, start, end = best_alignment

    print(format_alignment(*best_alignment))

    # Plotting matches
    matches = []
    positions = []

    for idx, (base1, base2) in enumerate(zip(aligned_seq1, aligned_seq2)):
        if base1 == base2 and base1 != '-':
            matches.append(1)
        else:
            matches.append(0)
        positions.append(idx)
```

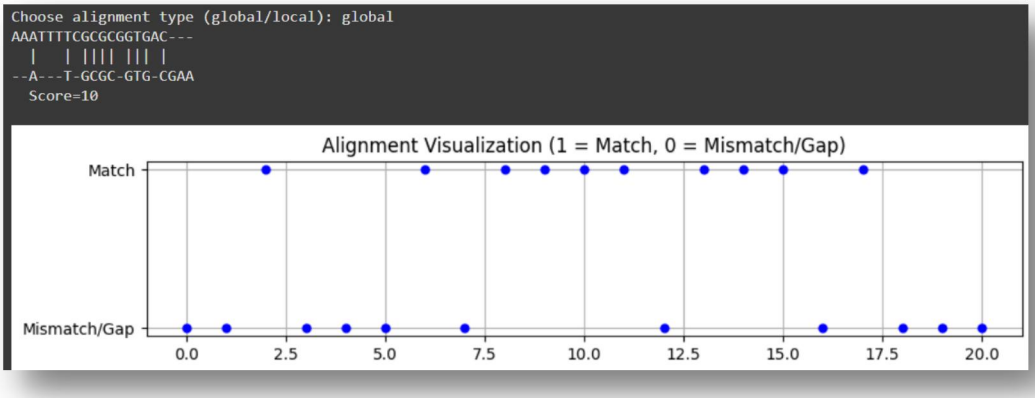
```python
plt.figure(figsize=(10,2))
plt.plot(positions, matches, 'bo', markersize=5)
plt.title("Alignment Visualization (1 = Match, 0 = Mismatch/Gap)")
plt.yticks([0,1], labels=["Mismatch/Gap", "Match"])
plt.grid(True)
plt.show()
```



**Full Analysis Mode:**

- When user selects Option 7, the program automatically:
  - Calculates and prints GC content.
  - Prints the reverse complement.
  - Prints the transcribed RNA.
  - Prints the translated protein sequence.
  - Displays the base composition pie chart.
  - Asks for alignment type and shows alignment.






## Special Features:

- **Dynamic Menu:** Users can perform multiple analyses without restarting the program.
- **Error Handling:** Handles invalid inputs and wrong choices gracefully.
- **Clear Visualization:** Base composition pie charts and alignment plots are neat and profession al.
- **User Flexibility:** Analyze either combined sequences or a single sequence based on user choi ce.

# Conclusion

This project successfully demonstrates the ability to manipulate, analyze, and visualize DNA sequences using Python. The project also lays a foundation for adding more complex bioinformatics workflows in the future.